



Escola Politécnica da PUCC

*Faculdade de Análise de Sistemas
Curso de Engenharia de Software*

Tópicos de Tecnologia e de Programação

**1º Semestre de 2026
Volume 1**

Prof. André Luís dos R.G. de Carvalho

Índice

ÍNDICE	2
CAPÍTULO I: O PARADIGMA IMPERATIVO OU PROCEDIMENTAL	3
INTRODUÇÃO.....	4
CARACTERÍSTICAS GERAIS.....	5
A INFLUÊNCIA DO MODELO VON NEUMANN	6
LINGUAGENS DE PROGRAMAÇÃO IMPERATIVAS	7
<i>Variáveis</i>	7
<i>Comandos</i>	9
<i>Subprogramas</i>	11
CONCLUSÃO.....	13
CAPÍTULO II: O PARADIGMA DECLARATIVO.....	15
INTRODUÇÃO.....	16
OBJETIVOS	16
CARACTERÍSTICAS	16
CONCLUSÃO.....	18

Capítulo I: O Paradigma Imperativo ou Procedimental

Introdução

Papert define paradigma de programação como sendo uma espécie de quadro estruturador subjacente à atividade de programar, que impactua diretamente na maneira pela qual o programador pensa sobre esta tarefa.

Pode-se dizer que os paradigmas de programação imprimem um estilo de representação de problemas, enquanto as linguagens de programação servem como meio de comunicar estas representações à máquina.

Inovações são coisas corriqueiras no mundo da computação; novas tecnologias estão constantemente emergindo. No entanto, a maioria destas inovações não pode ser ditas grandes inovações, no sentido de que, a maioria delas, simplesmente prove formas melhores para se fazer coisas que já se fazia.

Quando ocorre uma grande inovação, que provoca mudanças nos métodos básicos, na essência da forma pela qual uma tarefa é realizada, ou mesmo definida, dizemos que ocorreu uma mudança de paradigma.

Assim, mudar de paradigma de programação provoca muito mais impacto do que mudar de linguagem de programação. As implicações de uma mudança de paradigma de programação não se restringem à necessidade de vir a conhecer as estruturas sintáticas e semânticas de uma nova linguagem, engloba também mudanças no próprio processo de resolução de problemas.

Fica então patente a importância que se deve atribuir ao paradigma de programação, na medida que este é verdadeiramente um agente minimizador ou maximizador do esforço cognitivo a ser dispendido no aprendizado de uma linguagem de programação, podendo este ser facilitado sobremaneira, na medida em que as linguagens de programação encontrarem suporte em um paradigma adequado, refletindo-o e explicitando-o.

No começo, toda comunicação entre o homem e o computador era restrita a apenas uma dimensão. Tanto os comandos para o sistema operacional, como os próprios programas de computador, eram constituídos por linhas de texto que eram entradas via cartões perfurados ou via um terminal de fósforo branco.

Naquela época, classificar as linguagens de programação era uma coisa relativamente simples. O paradigma imperativo dominava o cenário e as aplicações se dividiam em científicas e comerciais. Quanto ao nível, as linguagens se dividiam em baixo nível (linguagem de máquina e linguagem de montagem) e de alto nível (Pascal, Fortran, Cobol, etc).

Esta situação permaneceu mais ou menos estável até por volta do ano de 1975, quando Smith propôs Pygmalion, proclamando uma nova era em termos de técnicas de programação, na qual o poder computacional das máquinas da época, juntamente com suas capacidades gráficas, passava a ser mais bem explorados e deixavam de servir como meros anteparos para linhas de texto.

Pode-se observar na década passada o acúmulo de evidências de que estas novas formas de programação traziam conseqüências realmente benéficas, facilitando o aprendizado de programação, aumentando a produtividade dos programadores e também seu grau de satisfação no desempenho de suas atividades.

A seguir, apresentaremos alguns destes paradigmas de programação, alguns dos quais já se firmaram entre os clássicos, enquanto outros ainda se encontram restritos às pesquisas acadêmicas.

Características Gerais

Também conhecido como procedimental, ou procedural, o paradigma imperativo é o mais antigo dos paradigmas. Em virtude disto, sua história se mistura muito com a história da computação propriamente dita.

Desde a primeira máquina de processar dados (uma simples máquina de somar construída por Pascal em 1642), até o primeiro computador com programa armazenado (concebido por John von Neumann e construído por Eckert e Mauchley em 1949), os computadores evoluíram muito conceitual e tecnologicamente.

Apesar da contínua e acelerada evolução tecnológica, durante muito tempo, a arquitetura dos computadores foi dominada pelo modelo de von Neumann, que ainda sobrevive e fundamenta a arquitetura da maioria dos computadores atuais.

A Influência do Modelo von Neumann

O paradigma imperativo de programação foi fortemente influenciado pelo modelo de computação de von Neumann. Ele teve um papel profundo na moldagem destas linguagens, que guardam, até hoje, uma estreita relação com a arquitetura dos computadores convencionais.

No modelo de von Neumann, programar adquire o significado de dar ordens ao computador, que as aceita e executa seqüencialmente. A memória e o processador são os dois principais módulos do modelo.

A memória é composta por um grande número de células numeradas, onde se podem armazenar valores. Ela serve, não só para armazenar os dados que um programa manipula, como também para armazenar instruções para o processador.

O processador, por sua vez, provê operações para modificar o conteúdo das células de memória. O número de uma célula de memória é comumente chamado de endereço, e serve para referenciar uma célula, tornando possível o acesso ao valor que ela armazena.

Apesar de atualmente atenderem aos mais diversos propósitos, foram as aplicações numéricas que impulsionaram o desenvolvimento dos computadores, o que explica o papel central que o sub-sistema de memória têm nesta arquitetura.

Os primeiros computadores, além de terem uma série de problemas decorrentes da tecnologia da época, eram extremamente difíceis de serem programados. A inexistência de programas de apoio à programação fazia com que toda programação fosse feita diretamente em código de máquina.

Este modo de programar levava à construção de programas absolutamente ilegíveis, o que tornava extremamente complicado o processo de localização e correção de erros de programação.

Esta foi uma forte motivação para a criação, não só dos montadores e das linguagens de montagem, como também das primeiras linguagens de alto nível, que surgiram trazendo novidades muito convenientes para as aplicações numéricas da época, tão carentes de certas facilidades ainda não disponíveis nas máquinas de então, como por exemplo, números reais, e acesso a elementos de uma seqüência por seu índice.

As primeiras linguagens de alto nível eram imperativas, e foram criadas de forma a imitar o mais perfeitamente possível as ações de baixo nível dos computadores da época.

Paralelamente ao desenvolvimento destas primeiras linguagens (como é comum acontecer com os paradigmas de programação) desenvolveu-se o paradigma imperativo, que de todos, é o paradigma que mais se aproxima do modelo de computação de von Neumann.

Linguagens de Programação Imperativas

Numa linguagem imperativa, escrever um programa para resolver um determinado problema é uma atividade que envolve especificar uma série de operações que, quando executadas em seqüência pela máquina, conduzem à solução do problema.

Desta forma, pode-se entender um programa como uma espécie de prescrição da solução de um problema.

Variáveis

Com as linguagens de programação imperativas, surgiu o conceito de variável, que é, talvez, um dos conceitos mais importantes introduzidos por estas linguagens. Variáveis são a abstração de um conjunto de células de memória e são identificadas por nomes.

Sua presença é marcante no contexto das linguagens imperativas, o que pode ser facilmente percebido ao se programar em uma delas, situação em que, normalmente, se depara o tempo todo com a necessidade de armazenar, recuperar e manipular dados localizados em variáveis.

Variáveis podem ser caracterizadas por uma sêxtupla de atributos: (Nome, Escopo, Sobrevida, Tipo, Endereço, Valor).

Embora existam variáveis sem nome, a maioria das variáveis possui um nome. O escopo de uma variável determina em que partes do programa a variável é visível, ou em outras

palavras, em que partes do programa ela pode ser referenciada por seu nome. A sobrevida de uma variável determina quando uma variável existe e retém o seu valor, e quando deixa de existir.

Os conceitos de escopo e sobrevida são muito comumente confundidos, apesar de existir uma clara diferenciação entre eles.

Quando uma determinada parte de um programa está sendo executada, uma certa variável, cuja sobrevida indica que ela existe, pode ou não poder ser referenciada por seu nome, dependendo de se estar ou não em seu escopo.

Por outro lado, é claro que uma variável, cuja sobrevida indica que ela não existe, não pode ser referenciada por seu nome, independentemente da parte do programa que está sendo executada.

O tipo de uma variável determina quais valores são válidos para serem armazenados em suas células, bem como, o conjunto de operações que podem operar sobre seu valor.

Quando uma variável pode ser referenciada pelo nome, este determina seu endereço, que, juntamente com seu tipo, serve para referenciar suas células de memória, de modo a tornar possível a manipulação do valor que elas armazenam.

O conceito de tipo desempenha um papel importante no contexto das linguagens imperativas, constituindo talvez o mais importante dos atributos de uma variável. Os tipos que uma linguagem implementa determinam em grande parte o estilo e o uso que se fará dela, e produz impacto profundo no processo de desenvolvimento de programas como um todo.

Assim sendo, é importante encontrar em uma linguagem uma variedade adequada de tipos e estruturas. Dentre os principais tipos de dados presentes nas linguagens imperativas de uso corrente, podemos destacar os tipos: lógico, inteiro, intervalo, enumerado, real, caractere, cadeia de caracteres, vetor, registro, união, conjunto e ponteiro.

Conhecer a dinâmica temporal do estabelecimento e rompimento das ligações entre variáveis e seus atributos é essencial para a compreensão da semântica das linguagens de programação.

Diz-se que uma ligação é dinâmica, quando pode ser estabelecida e rompida durante a execução do programa, e estática, quando isto não acontece. Em geral, ligações dinâmicas significam maior flexibilidade na programação, mas também comprometem a legibilidade, a eficiência e a confiabilidade do programa.

Comandos

As construções elementares de uma linguagem de programação imperativa são chamadas de comandos, e constituem a abstração de uma série de instruções de máquina.

Apesar da importância histórica que receber, processar, e produzir valores tem no contexto das linguagens imperativas, não se espera que cada comando individualmente produza um valor, mas sim que produza um efeito, ou, mais especificamente, o efeito de manipular valores de variáveis.

Nestas linguagens, via de regra, o armazenamento de valores em variáveis não tem uma importância fechada em si, mas sim uma importância contextual, na medida que impacta, diretamente, na atividade de outros comandos, que recuperam e usam o valor daquelas variáveis.

Desta forma, pode-se dizer que os comandos, em um programa imperativo, guardam uma estreita relação entre si e se combinam, usando um o efeito do outro, para alcançar os resultados que se deseja.

O Comando de Atribuição

Apesar das importantes abstrações promovidas pelas linguagens de programação imperativas, a manipulação de dados guardados em memória continua a ter um papel fundamental, e é feita, nestas linguagens, através do comando de atribuição.

Este comando tem ligação direta com a necessidade que se tem, na arquitetura von Neumann, de armazenar todo valor que for calculado, o que justifica a importância que ele tem em todas as linguagens imperativas, fazendo com que, de certa forma, o conceito de baixo nível de células de memória esteja presente em todas elas.

Esta presença subliminar força o programador a uma forma de pensamento totalmente vinculada a detalhes desta arquitetura, e assim, muito embora não se tenha nenhum interesse

direto na dinâmica da mudança de valores das células da memória, ou das variáveis, se preferirmos, ele tem que raciocinar nestes termos.

Estruturas de Controle

Embora toda computação em um programa imperativo seja feita pela avaliação de expressões e atribuição dos resultados a variáveis, em geral programas compostos tão somente por atribuições não apresentam muita utilidade.

Para possibilitar a escrita de programas interessantes, são necessárias estruturas de controle, que são comandos que permitem o controle do fluxo de execução dos programas.

Estruturas de controle consistem basicamente de um comando de controle e um conjunto de comandos que ele controla. Podem ser divididas nas seguintes categorias: comandos de seleção, comandos iterativos, e desvio incondicional.

Comandos de seleção permitem a escolha de um dentre dois ou mais caminhos de execução em um programa. Constituem de forma fundamental de toda linguagem de programação imperativa. Os tipos mais comuns de estruturas de seleção são: os seletores de uma condição, os seletores de duas condições, e os seletores de múltiplas condições.

Comandos iterativos são comandos que causam a repetição de um conjunto de comandos. Como os comandos de seleção, são essenciais em toda linguagem de programação imperativa.

Isto é consequência direta do modelo de computação de von Neumann, que prevê que as instruções que compõem um programa fiquem armazenadas na memória, juntamente com seus dados.

Se não existissem estruturas repetitivas, os programadores teriam que especificar todas as ações de um programa em seqüência, o que tornaria imenso qualquer programa útil, e inviabilizaria seu armazenamento na memória.

Além disto, o esforço de programação dispendido na escrita de um programa como este seria muito maior, e ainda assim, o resultado seria um programa pouco flexível, e, por isto mesmo, extremamente sujeito a mudanças.

Os tipos mais comuns de estruturas repetitivas são: as controladas por contador, as controladas por expressão lógica, as controladas por intervenção do programador, e os iteradores. Dentro de cada um destes tipos, as variações em forma dos comandos iterativos são inúmeras.

Um comando de desvio incondicional, ou, mais popularmente, um *goto*, transfere o fluxo de execução de um programa para um lugar específico dentro dele.

Trata-se de um comando tão poderoso, que todas as estruturas de controle podem ser simuladas com um seletor de uma condição e um comando de desvio incondicional.

Justamente esse poder é o que o torna perigoso. Sem um uso disciplinado, ele pode tornar os programas totalmente ilegíveis e, conseqüentemente, não confiáveis e difíceis de serem mantidos.

Muitas discussões sobre a manutenção da sua presença nas linguagens imperativas de alto nível e sobre restrições ao seu uso já ocorreram, sempre controvertidas e com muita polêmica.

Dijkstra foi o primeiro a produzir por escrito uma longa argumentação a respeito dos perigos do *goto*. Durante os anos que seguiram esta publicação, muitas pessoas defenderam, se não seu definitivo banimento das linguagens de alto nível, pelo menos severas restrições a seu uso.

Dentre aqueles que se posicionaram contrários à completa eliminação do comando *goto* destaca-se Knuth, que argumentava que existiam situações em que a eficiência deste comando suplantava qualquer dano que ele pudesse causar à legibilidade ou à manutenibilidade de um programa.

E apesar de toda controvérsia que gira em torno deste comando, ele sempre esteve e está presente na vasta maioria das implementações de linguagens de programação imperativas.

Subprogramas

Subprogramas estão entre os conceitos mais importantes do paradigma imperativo. Eles aumentam a legibilidade dos programas pela evidenciação de sua estrutura lógica, ao mesmo tempo em que esconde detalhes menores em um plano inferior.

A definição de um subprograma descreve as ações que estão sendo abstraídas nele, ao passo que a chamada de um subprograma é o pedido explícito da execução dessas ações.

Durante o período de tempo compreendido entre a chamada de um subprograma e a conclusão de sua execução, diz-se que um subprograma está ativo.

O cabeçalho de um subprograma é a primeira linha de sua definição, e serve a três propósitos básicos: (1) especificar que a unidade sintática que vem a seguir é um tipo de subprograma; (2) dar um nome ao subprograma; e (3) especificar, opcionalmente, uma lista de parâmetros daquele subprograma.

Os parâmetros especificados no cabeçalho de um subprograma são chamados de parâmetros formais. Comportam-se como variáveis que adquirem vida quando da ativação do subprograma, perdendo-a por ocasião de sua conclusão.

A chamada de um subprograma deve incluir o seu nome, bem como uma lista de parâmetros para ser associada aos parâmetros formais do subprograma. Os parâmetros especificados na chamada de um subprograma são chamados de parâmetros reais.

Em praticamente todas as linguagens imperativas, a associação entre os parâmetros formais e os reais é posicional, muito embora existam linguagens imperativas com outros mecanismos associativos.

Há vários mecanismos de passagem de parâmetros. Eles especificam o efeito que a atribuição de um valor para um parâmetro formal terá sobre o parâmetro real a ele associado.

Embora existam linguagens de programação imperativas que implementam mecanismos exóticos e excêntricos de passagem de parâmetros, e.g., passagem por resultado, por valor resultado e por nome, os mecanismos mais comumente encontrados nas linguagens de programação imperativas são as passagens por valor e por referência.

Existem duas categorias distintas de subprogramas: os procedimentos e as funções.

Procedimentos podem produzir resultados sensíveis pela unidade de programa chamante pela produção de efeitos colaterais, ou seja: (1) alterando parâmetros formais que permitam a transferência de dados ao chamante; e (2) alterando variáveis visíveis a ambos.

Funções se inspiram nas funções matemáticas. Se forem modelos fiéis, não produzirão efeitos colaterais, i.e., seu resultado será percebido como se efetivamente ele substituísse sua chamada, e não como são percebidos os resultados dos procedimentos.

Raras são as linguagens imperativas que implementam funções de forma fiel ao modelo matemático.

Conclusão

Pode-se dizer que programar em uma linguagem imperativa significa repetidamente computar valores de baixo nível e armazená-los em posições de memória.

É claro que este não é o nível de detalhe em que se deseja programar uma aplicação grande, e por isso muitas tentativas têm sido feitas no sentido de tentar ocultar a natureza de baixo nível da máquina.

Um resultado significativo destas tentativas é as expressões. Através delas o programador é capaz de comandar a produção de valores sem ter que se preocupar e nem mesmo notar a existência da produção e armazenamento de valores intermediários.

É importante ressaltar que, mesmo as expressões, que tanto colaboraram na camuflagem da natureza real da máquina, perdem suas propriedades matemáticas na presença dos efeitos colaterais.

Outro bom exemplo da influência da arquitetura da máquina nas linguagens imperativo é o conceito de subprograma.

Uma das maiores dificuldades em programar em uma linguagem imperativa reside nos testes e na correção dos erros de programação.

A corretude de um programa é determinada pela corretude de suas variáveis; localizar erros envolve observar o andamento da execução de um programa, executando mentalmente suas repetições, e observando a cada passo, o conteúdo de uma série de variáveis, o que é extremamente tedioso em programas grandes.

É certo que o escopo das variáveis, de certa forma, limita o espaço de observação. Mas se por um lado o conceito de subprograma é interessante por permitir a construção de componentes de programas independentes, por outro lado ele acaba revelando a natureza da máquina quando produzem efeitos colaterais, além de tornar complicado o processo de teste e depuração de programas.

Construções como estas existem nas linguagens imperativas em nome da eficiência, e a eficiência está ligada às características da arquitetura von Neuman.

Isto nos faz compreender melhor o que quis dizer Backus quando afirmou que a próxima revolução na programação somente ocorreria quando aparecesse um novo tipo de linguagem de programação, muito mais poderoso do que as linguagens atuais, e cujos programas pudessem ser executados a custos não muito mais altos do que os de hoje.

Capítulo II: O Paradigma Declarativo

Introdução

Sabemos que o paradigma imperativo coloca o programador na posição de alguém que manda (e, naturalmente, espera ser obedecido) e cujas ordens se agrupam para descrever o processo de atingir a solução de um determinado problema.

O paradigma declarativo tem esse nome porque, diferentemente do paradigma imperativo, coloca o programador na posição de alguém que declara o que é resolver um determinado problema, e não como resolvê-lo.

Surge com ele a chamada quarta geração de linguagens. E vale lembrar que enquadrámos na primeira geração de linguagens as linguagens de máquina, na segunda geração de linguagens as linguagens de montagem, e na terceira geração de linguagens as chamadas linguagens de alto nível, ou, em outras palavras, a grande maioria das linguagens de uso corrente, como por exemplo, Pascal, C, Fortran, Cobol, Clipper, entre outras.

Objetivos

O paradigma declarativo tem como principal objetivo melhorar o tempo despendido na geração de um programa, possibilitando maior produtividade dos programadores já que traz a possibilidade de uma programação de altíssimo nível.

Sabemos que quase todas as coisas têm um lado bom e um lado mal. O mesmo se dá com o levantamento do nível das linguagens. A simples observação das gerações de linguagens nos basta para, extrapolando, imaginar as implicações desta quarta geração de linguagens. São elas: (1) linguagens específicas e apropriadas para um domínio restrito de aplicações; e (2) linguagens que geram programas que cada vez menos uso eficiente da máquina fazem e que exigem para terem um tempo de execução aceitável, máquinas de mais alto desempenho.

Características

Como já sabemos, o paradigma declarativo coloca o programador na posição de alguém que declara o que é resolver um determinado problema, e não como resolvê-lo.

Outras aplicações também permitem a especificação de um problema em um nível alto de abstração, ou, em outras palavras, num nível em que fica caracterizado o que fazer e não como fazer. Ferramentas CASE são um exemplo do que falamos.

Só que nesse tipo de ferramenta, que tem apenas o compromisso de produzir uma especificação, e não o de produzir um programa executável, não existe uma preocupação exagerada com questões de precisão e de não ambigüidade.

Com as linguagens declarativas as coisas se passam de forma diversa: as especificações feitas nelas precisam poder ser executadas, e por isso precisam ser precisas.

Uma das linguagens mais precisas e menos ambíguas que há é a matemática. Daí vem a tendência que toda linguagem declarativa tem de fundamentar-se na matemática. Um exemplo do que dizemos é PROLOG, uma linguagem declarativa que se fundamenta em lógica formal (dizemos que segue o paradigma lógico). Outro exemplo é LISP, uma linguagem declarativa que se fundamenta em funções matemáticas (dizemos que segue o paradigma funcional). Um último exemplo é SQL, uma linguagem declarativa que se fundamenta em álgebra relacional.

As linguagens declarativas exibem propriedades muito interessantes. Uma delas é a transparência referencial. Em linguagens que têm transparência referencial o conceito de variável inexistente ou difere profundamente do conceito convencional no sentido de que, mesmo que existam variáveis em um programa, estas não são controladas explicitamente pelo programador na sua programação, e sim pelo próprio ambiente de execução da linguagem.

Outra propriedade é a transparência de fluxo de execução, ou, em outras palavras, o caminho da execução do programa é transparente para o programador.

Essas características são muito interessantes porque tornam independentes as partes constituintes dos programas, o que leva as linguagens declarativas a serem inerentemente paralelas, ou seja, nelas, construir um programa paralelo é absolutamente natural, tão natural que o programador nem precisa intervir de forma explícita, a paralelização dos processos pode ser feita de forma automática pelo próprio ambiente de execução da linguagem.

Atente para a importância do que dizemos. Com a tecnologia de hardware evoluindo, com a surgimento das máquinas com múltiplos processadores, a existência de linguagens que possibilitam a criação de programas paralelos de forma tão natural e simples é de extrema relevância.

Conclusão

O paradigma declarativo não é um paradigma novo. Conhece-se linguagens que o representam há pelo menos 40 anos. Apesar de ser um paradigma antigo, podemos dizer sem medo de errar que, nestes 40 anos, as linguagens declarativas jamais alcançaram níveis consideráveis de uso em aplicações reais. Uma exceção talvez seja a linguagem SQL e outras linguagens de consulta.

A colocação acima poderia nos levar a questionar: porque afinal estudamos linguagens declarativas? A resposta é simples! Apesar de antigas, as linguagens declarativas não alcançaram a preferência dos programadores de sistemas, não por serem desinteressantes, e sim por serem muito ineficientes computacionalmente, enquanto as linguagens imperativas, por exemplo, não apresentam essa desvantagem.

A maior eficiência das linguagens imperativas pode ser explicada pela íntima relação que guardam com a arquitetura das máquinas convencionais, a chamada arquitetura von Neuman. Não exploraremos com maior vagar a questão desta relação neste texto por fugir esta discussão do escopo deste texto.

Procuraremos discutir a seguir as seguintes questões: (1) Eficiência computacional é ainda um aspecto tão importante? (2) As linguagens declarativas sempre poderão ser consideradas ineficientes? (3) As linguagens declarativas são de fato artigos de museu e merecem ser esquecidas?

Começemos pelo princípio. Eficiência computacional é ainda um aspecto tão importante? Raciocinemos! A importância atribuída à eficiência computacional está relacionada a questão do interesse que se tem em aplicações computacionais que executem num tempo aceitável.

Mas, nas modernas máquinas de que dispomos na atualidade, mesmo aplicações muito ineficientes computacionalmente executam em um tempo bastante aceitável.

Vale lembrar ainda que o preço dessas máquinas cai vertiginosamente, enquanto o mesmo não acontece com o preço da mão de obra especializada. Desta forma, melhorar a produtividade é muito interessante, mesmo que isto custe o sacrifício da eficiência computacional.

Consideremos agora a questão seguinte. As linguagens declarativas sempre poderão ser consideradas ineficientes? Na verdade, a resposta para esta questão é não. As linguagens declarativas são ineficientes em máquinas de arquitetura convencional, mas hoje em dia já existem arquiteturas que fogem deste convencional.

Sabemos que, em máquinas paralelas, as linguagens declarativas se tornam muito interessantes por duas razões. A primeira é que a velocidade das máquinas paralela torna aceitável o tempo de execução de programas declarativos. E a segunda é que as linguagens declarativas, por serem naturalmente paralelas, possibilitam explorar com mais naturalidade este tipo de máquina.

Linguagens declarativas e máquinas paralelas formam um par perfeito, assim como as linguagens convencionais formam um par perfeito com as máquinas convencionais.

Assim sendo, a resposta para nossa terceira questão parece simples. As linguagens declarativas são de fato artigos de museu e merecem ser esquecidas? A resposta claramente é não! Podemos dizer que as máquinas de quinta geração vão colocar as linguagens declarativas em seu próprio tempo.
